Epistract: A Two-Layer Knowledge Graph Framework with an Epistemic Super-Domain Layer

The research-consolidation layer for Claude Code:
turning a body of deep research into a structured, queryable graph project

Umesh Bhatt
umesh@8thcross.com
@8thcross on Medium

with contributions from Chris Davidson (clinicaltrials, fda-product-labels)

April 2026 (framework v3.2.0)

Abstract

Most knowledge graph construction tools stop at extracting entities and typed relations: they tell you what a corpus says, but not *how* the corpus says it. A relation between a compound and a disease in a peer-reviewed clinical paper carries different epistemic weight from the same relation in a forward-looking patent claim, yet flat knowledge graphs collapse the distinction. I present Epistract, an open-source domain-agnostic knowledge graph framework distributed as a Claude Code plugin. Claude Code already provides rich deep-research capabilities – built-in web tools plus a growing ecosystem of plugins and MCP servers for academic literature search, patent and trial-registry retrieval, regulatory database access, and browser automation. Epistract is positioned *downstream* of that deep-research stage: it is the tool that consolidates a body of documents (assembled through deep research, manual curation, or both) into a structured knowledge graph project from which to answer a bundle of specific questions. The technical contribution is a *two-layer* graph: Layer 1 contains the brute facts (entities, typed relations, evidence quotes, confidence scores), and Layer 2 is an *epistemic super-domain layer* that annotates every relation with a categorical status (`asserted`, `prophetic`, `hypothesized`, `contested`, `contradiction`, `negative`, `speculative`) using a rule engine combining regex patterns over evidence text with document-type inference. A single domain-specific analyst persona, defined as one YAML string, drives two complementary surfaces: a reactive chat interface over the graph, and a proactive auto-generated briefing that synthesizes the classified graph into temporally-stratified, contradiction-aware analyst output. The graph that results is bespoke to the consolidated body: a different question-bundle, with a different corpus and possibly a different domain, produces a different graph – the framework's value is not in producing one canonical KG of the world, but in supporting the construction of many specialized KGs grounded to specific bodies of deep research. The framework ships with four pre-built domains: drug-discovery (validated across six scenarios spanning neurogenetics, oncology, rare disease, immuno-oncology, cardiovascular inflammation, and GLP-1 competitive intelligence – 111 documents, 783 nodes, 2,230 relations across S1–S5 plus 278 nodes / 855 relations on S6 alone after a v3 rebuild), clinicaltrials (validated on a GLP-1 Phase 3 corpus from `ClinicalTrials.gov` – 10 protocols, 142 nodes, 395 relations), contracts, and fda-product-labels (the latter two are schema scaffolds awaiting their first showcase corpus). Each domain carries its own epistemic rules: phase-based evidence grading for clinicaltrials, a four-level FDA epistemology classifier (established / reported / theoretical / asserted) for fda-product-labels. Adding a new domain requires no pipeline changes – it is four configuration files (schema, extraction prompt, epistemic rules, persona) generated interactively via `/epistract:domain`. The framework, all corpora, and all artifacts are MIT-licensed and reproducible.[1]

[1] https://github.com/usathyan/epistract

1 Motivation: the missing dimension

Modern knowledge graph (KG) construction has converged on a recognizable pattern. Documents enter; an extractor (whether rule-based, fine-tuned NER, or LLM-driven) emits entities and typed relations; a graph builder deduplicates and resolves; a visualizer renders. The output is a labeled property graph that answers structural questions: which entities are most central, which clusters form, which relations are most prevalent. The flagship knowledge-graph platforms – Neo4j Bloom, Stardog, GraphDB, Microsoft's GraphRAG Edge et al. [2024], and a growing ecosystem of open-source LLM-based extractors – all deliver this primary product.

A flat knowledge graph, however, is fundamentally lossy with respect to the epistemic content of its source corpus. Consider three sentences that might appear in a drug-discovery corpus:

1. "In SURPASS-2, tirzepatide 15 mg reduced HbA1c by 2.30% versus 1.86% for semaglutide 1 mg (p<0.001)." Frías et al. [2021]

2. "In one or more embodiments, the compositions of the present invention may be useful for the treatment of cardiovascular disease, neurodegenerative disorders, and metabolic dysfunction."

3. "GLP-1 receptor agonism has been hypothesized to confer neuroprotective effects through mechanisms that are not yet fully characterized."

All three sentences are about drug-disease relationships, and a typed-relation extractor (the dominant pattern in modern KG construction) will produce structurally similar edges from each. To make the comparison concrete, take sentence (1). A traditional flat KG – whether built by Neo4j-with-spaCy, GraphRAG, or any of the open-source LLM extractors – would yield approximately the following from this single sentence:

> **Entities.** `tirzepatide` (Compound), `semaglutide` (Compound), `HbA1c` (Biomarker), `SURPASS-2` (ClinicalTrial), `type 2 diabetes` (Disease, often inferred from HbA1c context).
>
> **Typed relations.**
> `tirzepatide REDUCES HbA1c` (conf 0.95)
> `semaglutide REDUCES HbA1c` (conf 0.95)
> `tirzepatide EVALUATED_IN SURPASS-2` (conf 0.99)
> `semaglutide EVALUATED_IN SURPASS-2` (conf 0.99)
> `tirzepatide INDICATED_FOR type_2_diabetes` (conf 0.85)
> `semaglutide INDICATED_FOR type_2_diabetes` (conf 0.85)
>
> **Edge metadata available.** Confidence score; verbatim evidence quote; source document identifier.

This is a perfectly serviceable labeled property graph. It is typed and directional, it carries source attribution, and it is queryable: "which compounds are indicated for type-2 diabetes?" returns both. What it cannot tell the analyst is that this answer comes from a Phase 3 head-to-head with $p < 0.001$. Querying the same flat graph after sentence (2) is also ingested would return identical-shape edges asserting tirzepatide is indicated for cardiovascular disease and neurodegenerative disorders – claims that are patent-prophecy, not empirically established. Confidence is a poor discriminator: a thorough patent extractor will produce `prophetic` edges with high confidence (the patent really does claim them), and a hedged review extractor will produce `hypothesized` edges with lower confidence – but the confidence axis collapses the categorical distinction. Sentence (2) yields edges with status *prophetic* (rule: "in one or more embodiments…" phrase + document-type-is-patent inference); sentence (3) yields *hypothesized* (rule: "has been hypothesized" regex). Two graphs containing nominally identical edges produce different decisions for an analyst reading them.

This is not a problem that domain-specific extraction prompting solves. The information is present in the source text – the syntax of patents, hedging vocabulary in research papers,

the structural conventions of trial reports – but a graph that records only ⟨ subject, predicate, object, confidence, source ⟩ has no field in which to store it. The missing dimension is *epistemic typing*: a categorical label, attached to each relation, that records what kind of claim it is.

Who the framework is for, and what it is for. Claude Code already provides rich *deep-research* capabilities: built-in web search and document fetch, plus a growing ecosystem of plug-ins and MCP servers for specialized research workflows – academic literature search, patent and regulatory database access, trial-registry retrieval, competitor and market intelligence, browser automation, third-party data stores. A scientist or computational analyst using Claude Code can already conduct deep research on a topic: collect papers, fetch patents, pull trial protocols, scrape regulatory filings. Epistract is positioned *downstream* of that deep-research stage. Once a body of documents has been assembled – through Claude Code's deep-research tools, manual curation, an institutional document store, or any combination – Epistract is the tool that consolidates that body into a structured knowledge graph project: typed entities and relations grounded in the domain's ontology, categorical epistemic statuses on every relation, an analyst briefing, and a reactive chat surface. The graph project then becomes the substrate for answering many specific questions about the consolidated body. The framework is built for biomedical researchers, clinical-trial analysts, regulatory affairs teams, competitive-intelligence professionals, contract analysts, and others whose work involves rigorous synthesis of literature sets they have already assembled or are continuing to assemble through deep research.

It is primarily an *analytical* tool: it does not transact, does not write back to the source corpus, and does not generate new documents. It produces a knowledge graph, an epistemic classification of every relation in that graph, and an analyst briefing. The scientist consumes those artifacts and asks follow-up questions in the workbench chat. The framework's job is to accelerate the synthesis-and-analysis phases of scientific work – consolidating evidence, cataloging claims by epistemic status, surfacing contradictions and gaps, supporting follow-up question-answering – while keeping deep research and analytical decision-making in the scientist's hands.

Bespoke consolidation, per question-bundle. The graph that Epistract produces is *bespoke*: it is shaped by the corpus a scientist assembled to answer a question or a bundle of related questions. "How does the contemporary GLP-1 receptor agonist competitive landscape look across Phase 3 outcomes?" is one such question-bundle – and the deep research that answers it might span twenty-four PubMed papers, ten patent filings, and ten ClinicalTrials.gov protocols, assembled over hours or days. Epistract's role begins after that body is in hand. The framework consolidates the body into a graph whose entity types come from the chosen domain's schema (drug-discovery, clinicaltrials, or a domain the scientist extends via `/epistract:domain`), whose relations carry both confidence and categorical epistemic status, and whose persona-driven narrator surfaces the patterns – prophetic concentrations, contested edges, gap signals – that an analyst would otherwise have to find by hand. The same body, a different question-bundle, would call for a different consolidation pass and possibly a different domain.

Iterative refinement. A scientist's understanding of a consolidated body is not a one-shot phenomenon. As they read the briefing and explore the graph, they discover gaps (a missing study, an unindexed patent), refinements to the analytical frame (a new entity type to track, a hedging pattern the rules missed, a contradiction worth promoting to a custom rule), and follow-up questions worth chasing. Some of those follow-ups send the scientist back to deep research – using Claude Code's web tools or MCP-mediated source connectors to acquire missing documents – and back to Epistract for reconsolidation. The framework supports this loop as discrete commands: `/epistract:ingest <new-folder>` appends documents to the corpus and re-extracts; `/epistract:epistemic` re-runs Layer 2 classification and the narrator briefing against the updated graph; the workbench chat reactively answers follow-ups against the current

graph state. The framework itself does not auto-learn across runs – refinements are human-mediated, and GitHub Issue #15 tracks the aspirational compounding mechanism. What does persist across runs is the *scientist's* broader context: Claude Code's built-in memory system and its plugin / MCP ecosystem keep the accumulated knowledge – what they have tried, what they have found, what they want to try next – available across sessions. Deep research lives upstream of the consolidation; the consolidation feeds new questions; new questions drive more deep research. Epistract is the consolidation layer in that loop.

Returning to the architecture. This paper introduces Epistract, a domain-agnostic knowledge graph framework whose primary contribution is to treat the epistemic dimension as a first-class architectural layer rather than as ad-hoc per-relation metadata. Epistract produces *two-layer* graphs: Layer 1 is the conventional brute-facts graph (entities, typed relations, evidence quotes, confidence scores – exactly what the worked example above shows the flat KG producing), and Layer 2 is an *epistemic super-domain layer* that annotates every relation with a categorical status using a rule engine combining regex patterns over the verbatim evidence text with document-type inference. The two layers are produced in the same pipeline, persisted in the same artifact (`claims_layer.json`), and rendered together in the visualization. A separate analyst-persona stage consumes the classified graph (not the source text) to produce a proactive briefing – contradictions surfaced, prophetic claims separated from asserted ones, follow-up actions recommended.

Two design choices distinguish the framework from prior work. The first is *domain pluggability*: the pipeline is domain-agnostic, and a domain is defined by four small configuration files (schema in YAML, extraction prompt in Markdown, epistemic rules in Python, analyst persona in YAML). The framework ships with four pre-built domains – drug-discovery, contracts, clinicaltrials, and fda-product-labels – and adding a new one requires no Python changes to the core pipeline. The second is the *persona dual-use pattern*: a single YAML string per domain (the analyst persona) is the system prompt for both a reactive chat surface (workbench, where users ask ad-hoc questions) and a proactive briefing surface (the auto-generated narrator output). The two surfaces stay coherent because they derive from the same persona.

The remainder of this paper is organized as follows. Section 2 describes the two-layer architecture and the pipeline. Section 3 details the epistemic super-domain layer – the rule engine, the status taxonomy, and the persona-driven briefing. Section 4 describes the four pre-built domains and the `/epistract:domain` wizard for adding new ones. Section 5 reports validation across the seven scenarios spanning two active domains. Section 6 situates the framework relative to similarity-based KG construction. Sections 7–9 discuss collaboration, availability, and limitations.

2 Architecture: a two-layer pipeline

Epistract implements a six-stage pipeline (Figure 1). Documents enter; an agent-based extractor produces typed entities and relations against a domain schema; a deterministic validation step verifies molecular identifiers (drug-discovery only); a graph builder deduplicates and detects communities; an *epistemic classifier* annotates every relation with a categorical status; and an analyst-persona stage produces a proactive briefing from the classified graph. The visualizer surfaces both layers together, with an interactive chat panel grounded in the same domain persona.

2.1 Agent-based extraction

Extraction agents run as Claude Code subprocesses, not stateless API calls. Each agent owns one full document end-to-end, reads it with domain understanding informed by the domain's

`SKILL.md` extraction prompt, and produces a single `DocumentExtraction` JSON object conforming to the domain schema's entity and relation types. For corpora of four or more documents, agents execute in parallel, with one extractor per document. Each agent emits, per relation, a verbatim evidence quote and a confidence score in $[0, 1]$ – two fields the epistemic classifier in Layer 2 depends on.

Chunking strategy. Within each agent's lifetime, the document is read via sentence-aware chunking. The framework segments documents into chunks of up to 10,000 characters using `chonkie`'s `SentenceChunker` cho [2024] with a character tokenizer, with the final ∼1,500 characters (the last three sentences) of each chunk repeated as overlap into the next. Sentence boundaries are preserved – chunks never split mid-sentence – so phrases like *the same compound*, *this trial*, or *the previous result* resolve through the overlap window rather than evaporating at chunk boundaries. The sentence-aware chunker is the v3.0 contribution to this stage of the pipeline (project planning reference: Phase 14 / FIDL-03): earlier framework versions used naive character-position splits that cut sentences and dropped cross-chunk relations; the move to `chonkie` preserves sentence integrity and adds the explicit overlap window, measurably improving cross-chunk relation recovery on long patent claims and multi-section trial protocols where coreference frequently spans paragraphs. The chunk size and overlap are tuned to fit comfortably within Claude's context window with substantial headroom for the extraction prompt and schema – a single agent processes its document's chunks sequentially within one session, maintaining context across them.

Contrast with chunked-RAG and stateless extractors. This is qualitatively different from chunked-RAG retrieval. A chunked retriever decomposes the entire corpus into chunks indexed by embedding and retrieves them *across documents* by similarity to a query; cross-paragraph context within any one document is lost when only the matched chunks come back. Epistract's per-document agents instead read every chunk of one document sequentially within a single agent session, with the explicit overlap window between adjacent chunks. The discourse structure of each individual document – methods, results, discussion, hedging – survives intact, because one agent owns one document end-to-end. A stateless API extractor, processing chunks in isolation across separate calls, cannot resolve cross-chunk references the way an in-session agent can. An embedding retriever, operating across the entire corpus, never has the per-document discourse structure available to it in the first place.

2.2 Domain pluggability

Domain knowledge enters the pipeline through four files, all in a self-contained per-domain directory:

- `domain.yaml` – entity types and relation types in YAML, with names, descriptions, and example surface forms. The *schema*.
- `SKILL.md` – the extraction prompt the agent reads. Includes naming conventions (INN names for drugs, HGNC symbols for genes, MeSH preferred terms for diseases), confidence-scoring guidance, and per-entity-type examples.
- `epistemic.py` – the per-domain rule engine that produces categorical statuses. Includes `HEDGING_PATTERNS` (regex over evidence text), `infer_doc_type()` (inference rules from filename or text features), and any domain-specific custom rules (e.g., the four-level FDA epistemology classifier in `fda-product-labels/epistemic.py`).
- `workbench/template.yaml` – the analyst persona, in one YAML field as a multi-line string. This is the system prompt for both the reactive chat and the proactive narrator.

The core pipeline (`core/`) is domain-agnostic. `core/domain_resolver.py` loads a domain by name, locates the four files, and threads the schema, prompt, and rule module through the rest of the pipeline. Adding a domain is a purely declarative act: write the four files, place them under `domains/<name>/`, and `/epistract:ingest --domain <name>` works. The framework ships with a `/epistract:domain` wizard that bootstraps these four files from a sample corpus through a guided three-pass interactive flow.

2.3 Deterministic validation (drug-discovery only)

Drug-discovery alone among the four shipped domains carries a deterministic post-extraction validation step. SMILES strings are validated and canonicalized through RDKit Landrum [2024]; protein and nucleotide sequences are checked against standard alphabets through Biopython Cock et al. [2009]; structured identifiers (CAS registry numbers, NCT trial numbers, InChIKeys) are validated by regex and, where possible, cross-referenced. The validation report is auto-emitted as `validation_report.json` alongside the graph. The other three shipped domains do not require this stage because their entity types are not chemistry-bound; the framework simply skips the validation step when the domain provides no `validation-scripts/` directory.

2.4 Graph construction

Once extraction and (optional) validation are complete, sift-kg Ceresa [2024] handles graph assembly: entity deduplication via SemHash and Unicode normalization, Louvain community detection, and interactive HTML visualization via pyvis Perrone [2018]. A custom labeling engine assigns descriptive community names from member entity composition rather than numeric identifiers, with the labeling rules type-aware: gene-dominant clusters receive names reflecting the biological context of their gene members, pathway-driven clusters are named for the dominant pathway, and clusters organized around a mechanism–cell type axis receive compound labels. The output of this stage is `graph_data.json` – the canonical Layer 1 artifact.

2.5 Epistemic classification (Layer 2)

The new architectural element in v3 is a separate post-graph stage that annotates Layer 1 with categorical epistemic statuses. This is the subject of Section 3; in summary, the stage reads `graph_data.json`, applies the domain's rule engine (`epistemic.py`) to every relation's verbatim evidence and source-document type, and writes `claims_layer.json`. This artifact is the canonical Layer 2 output. It is reproducible (rule-based) and bit-identical across runs given identical Layer 1 input.

2.6 Persona-driven narration

A second post-graph stage consumes the classified graph (*not* the source documents) and produces a proactive briefing. The narrator's system prompt is the domain's analyst persona, loaded directly from `workbench/template.yaml`. Its input is a structured summary of the classified graph – per-status counts, contested-claim pairs, prophetic-claim concentrations, gap signals – assembled by `core/label_epistemic.py` from `claims_layer.json`. The output is `epistemic_narrative.md`, a domain-shaped analyst briefing with executive summary, contested claims, prophetic claims, gaps, and recommended follow-ups. Unlike Layer 2, this output is non-deterministic (LLM-generated); the deterministic record is `claims_layer.json`.

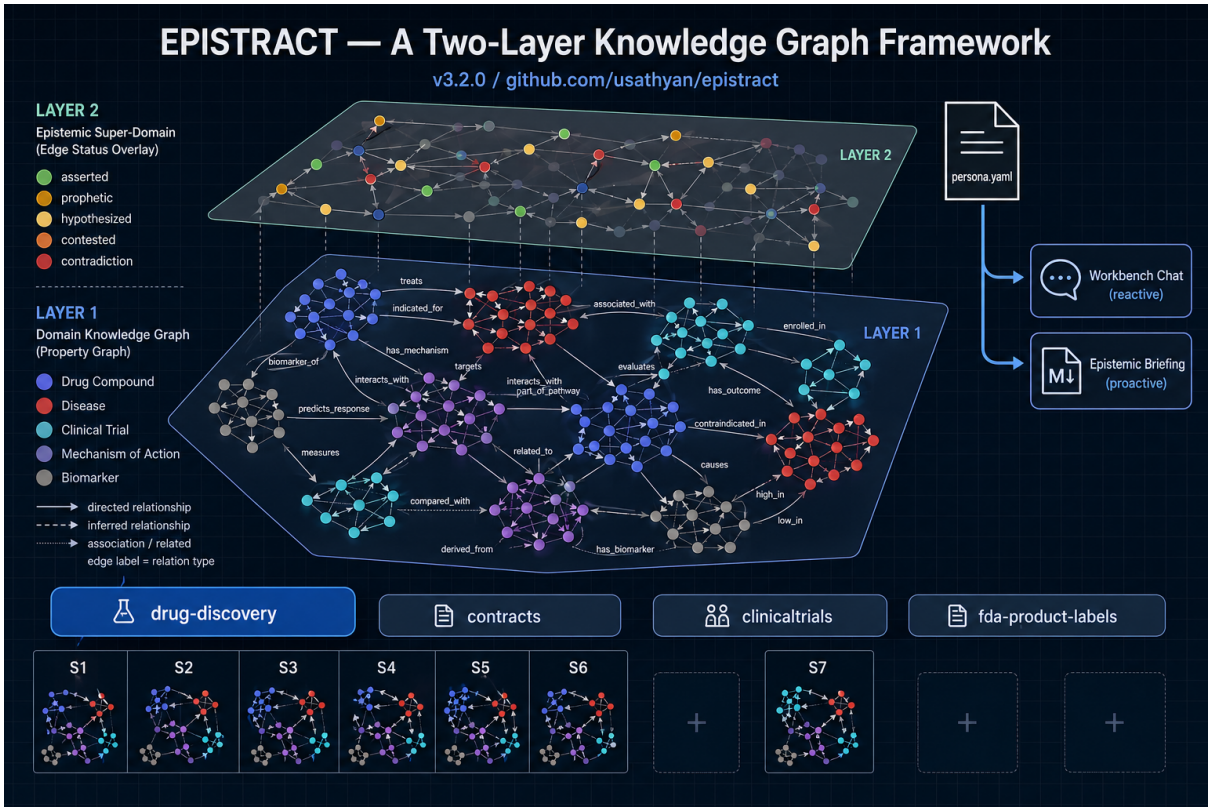

Figure 1: Epistract on one page. **Layer 1, the brute facts**: a force-directed labeled property graph with entities (compounds, diseases, mechanisms, biomarkers, clinical trials) extracted from a corpus and color-coded by entity type, connected by typed directional relations. **Layer 2, the epistemic super-domain**: a categorical status (`asserted`, `prophetic`, `hypothesized`, `contested`, `contradiction`, `negative`) annotated on every relation by a rule engine that combines regex patterns over evidence text with document-type inference. **The analyst persona** (right): a single `persona.yaml` string per domain drives both the reactive workbench chat and a proactive auto-generated briefing (`epistemic_narrative.md`) – one source of truth, two surfaces. **The four pre-built domains** (bottom): drug-discovery, contracts, clinicaltrials, fda-product-labels. **Track record** (S1–S7): scenario corpora that have pressure-tested each domain.

2.7 Workbench: the reactive surface

A web-based dashboard (the workbench, launched via `/epistract:dashboard`) renders the graph with a chat panel beside it. The chat's system prompt is the same analyst persona used for the narrator. Both surfaces – the auto-generated briefing and the chat – are grounded in the same graph data and the same persona. This is the persona dual-use pattern: one source of truth (`template.yaml`'s persona field), two surfaces (proactive narrator, reactive chat). Section 3 returns to this point.

2.8 Plugin delivery

Epistract is distributed as a Claude Code plugin Anthropic [2024]. Users install with `/plugin install epistract` and invoke pipeline stages via slash commands: `/epistract:ingest`, `/epistract:epistemic`, `/epistract:dashboard`. The plugin encapsulates the entire dependency tree (Python virtualenv, sift-kg, RDKit, Biopython, optional sentence-transformers) and exposes a single entry point per pipeline stage.

3 The Epistemic Super-Domain Layer and the Narrator

This section describes the framework's distinctive analytical layer: the categorical epistemic typing on every relation, the deterministic rule engine that produces it, the per-domain customization paths, and the persona-driven *narrator* stage that synthesizes the classified graph into an analyst briefing. Three output artifacts are produced together, and each has its own consumer:

- `graph_data.json` – Layer 1, the brute-facts graph (Section 2).
- `claims_layer.json` – Layer 2, the deterministic per-relation epistemic classification produced by the rule engine. Reproducible bit-identically across runs given identical Layer 1 input. The canonical machine-readable record.
- `epistemic_narrative.md` – the analyst briefing produced by the persona-driven narrator stage from the classified graph (*not* from the source documents). Structured into Executive Summary, Asserted Claims, Prophetic Claims, Contested Claims, Gaps, and Recommended Follow-Ups. Non-deterministic in word choice but stable in structure and findings across runs.

A fourth surface, the *workbench chat*, is the reactive consumer of the same classified graph and the same domain persona. Together, the classified graph (`claims_layer.json`), the proactive briefing (`epistemic_narrative.md`), and the reactive chat are what the framework adds beyond a typed-relation knowledge graph – and they are what an analyst actually consumes when working with the framework on a corpus.

3.1 Rationale

A confidence score in $[0, 1]$ on a relation is a one-dimensional signal: it conveys how strongly the extractor believes a given subject–predicate–object triple is in the source text. It does not convey what *kind of claim* the source text is making. The same edge in the graph could be supported by source language that asserts ($p < 0.001$ in a randomized trial), prophesies ("in one or more embodiments…" in a patent), hypothesizes ("has been suggested to…" in a discussion section), or denies ("did not show a statistically significant…"). These differ *categorically*, not by degree, and a scalar confidence flattens them all into the same numerical surface.

The epistemic super-domain layer adds a second dimension. Every relation in the graph receives a categorical *epistemic status* from a small fixed taxonomy. The taxonomy was chosen to be (a) source-language-driven (every status corresponds to identifiable patterns in the verbatim evidence text), (b) decision-relevant (analysts make different decisions about `prophetic` vs. `asserted` edges), and (c) cross-domain general (the same taxonomy applies in drug-discovery, contracts, clinicaltrials, and FDA labels, with per-domain variations).

3.2 The status taxonomy

The shared epistemic status taxonomy contains seven categorical labels (Table 1). Six are positive classifications (a relation has been classified as one of these); the seventh, `unclassified`, is assigned when no rule matches. Domain-specific epistemic modules may add or refine statuses (for example, `fda-product-labels` introduces a four-level classifier mapping onto these statuses with finer granularity).

3.3 The rule engine

Layer 2 is produced by a deterministic rule engine in `core/label_epistemic.py`, parameterized by the domain's `epistemic.py` module. The engine has three stages, applied per relation:

Table 1: The shared epistemic status taxonomy.

Status	Meaning and trigger patterns
`asserted`	Quantitative or definitive claim grounded in the source. Default for evidence text containing trial results, p-values, dose-response data, or unhedged assertion.
`prophetic`	Patent forward-looking language. Triggered by phrases such as "in one or more embodiments," "may be useful for," "the present invention provides," and document-type inference (the source document is a patent filing).
`hypothesized`	Hedged research language. Triggered by "hypothesized," "has been suggested," "may indicate," "is thought to," and similar epistemic hedging vocabulary.
`contested`	The same subject–predicate–object triple appears across multiple source documents with materially different confidence scores or contradictory evidence quotes. Detected at cross-source aggregation time, not within a single document.
`contradiction`	Explicit denial of a claim asserted in another source ("did not demonstrate," "failed to show statistical significance," "no evidence of…"). Stronger than `contested`.
`negative`	Explicit absence claim from a single source ("no association observed," "no detectable expression"). Distinct from `contradiction` (which is cross-source).
`speculative`	Marked future-conditional language outside patents ("further studies are needed," "the implications remain to be determined").
`unclassified`	No rule fired; default fall-through.

1. **Document-type inference.** The source document's filename and any cached document-type hint (e.g., "patent_WO2024_*_zealand.txt" triggers patent-type inference) drive a coarse classification. Patent-source relations default to `prophetic` unless a more specific trigger fires.

2. **Hedging-pattern matching.** A list of compiled regex patterns runs against the relation's verbatim evidence text. Each pattern carries a status it produces on match. Patterns are short, conservative, and audited: they prefer `unclassified` over a wrong classification.

3. **Cross-source aggregation.** After per-relation classification, the engine groups relations by ⟨subject, predicate, object⟩ identity and inspects the multi-source set. If the same triple appears with confidence ranges spanning more than 0.30 across sources, it is annotated `contested`. If a source's evidence text is a denial of another source's assertion, the deniers are annotated `contradiction`.

The output of the rule engine is `claims_layer.json`: a per-relation record with the assigned status, the matching rule (for audit), the verbatim evidence quote, and a back-reference to the source document. This artifact is bit-identical across runs given identical Layer 1 input. It is the reproducible Layer 2 record.

3.4 Domain-specific epistemic rules

Each domain's `epistemic.py` can add domain-specific rules beyond the shared taxonomy. Two examples illustrate the pattern.

Phase-based evidence grading (clinicaltrials). Clinical trial documents from `ClinicalTrials.gov` carry trial-phase metadata (Phase 1, 2, 3, 4). The clinicaltrials epistemic module promotes Phase 3 and Phase 4 outcome relations to `asserted`, leaves Phase 2 at the rule-engine default, and demotes Phase 1 outcomes to `hypothesized` unless the evidence quote explicitly asserts a quantitative result. This phase-based grading captures regulatory-grade evidence stratification that a domain-agnostic rule engine cannot infer from text alone.

Four-level FDA epistemology (fda-product-labels). FDA Structured Product Labels carry a recognizable epistemic vocabulary ("has been demonstrated," "has been reported," "has been theorized," implicit assertions in the Indications and Usage section). The fda-product-labels epistemic module implements a four-level classifier mapping onto the shared taxonomy with finer granularity: `established` (regulatory-grade asserted), `reported` (lower-tier asserted), `theoretical` (hypothesized in the FDA-specific sense), and a default `asserted` for unmarked indications-and-usage statements. The four levels are visible to the analyst persona, which can stratify briefings accordingly.

These per-domain extensions illustrate the framework's pluggability: the core pipeline does not need to know about phase-based grading or FDA epistemology vocabulary – the domain's `epistemic.py` encodes that knowledge, and `core/label_epistemic.py` dispatches to it.

3.5 Predefined defaults, customizable per domain

The framework ships with a usable epistemic layer out of the box. Every pre-built domain comes with: (a) the seven-status taxonomy in Table 1; (b) a starter set of hedging-pattern regex rules that fire on common research-language constructions across all biomedical and contract corpora; (c) document-type inference defaults that recognize patent filings, trial-protocol records, FDA labels, and peer-reviewed papers from filename and structural cues; and (d) the cross-source aggregation rule that detects `contested` edges when the same triple appears with confidence variance above a threshold across documents. A user who installs the framework, picks a pre-built domain, and ingests their corpus gets all four behaviors without further configuration.

These defaults are the floor, not the ceiling. Three customization paths are supported:

- *Add or refine hedging patterns.* The domain's `epistemic.py` exposes `HEDGING_PATTERNS` as a list of ⟨regex, status, rationale⟩ triples. Adding a new pattern is one line; the rule engine picks it up on the next ingest.
- *Add domain-specific status types.* The clinicaltrials and fda-product-labels domains demonstrate how a domain can introduce statuses beyond the shared seven (`established`, `reported`, `theoretical` for FDA labels) by extending the rule engine. New domains can do the same; the rule-engine dispatch is open.
- *Override the cross-source aggregation threshold or add custom rules.* A domain can plug in `CUSTOM_RULES` (cross-source contradiction patterns, domain-specific contested-pair detection, special-case escalations) without modifying `core/label_epistemic.py`. The wizard generates a starter `epistemic.py` pre-populated with the shared taxonomy and the pattern of `HEDGING_PATTERNS` / `CUSTOM_RULES` so authors edit rather than write from scratch.

The customization story is part of the workspace pattern. A scientist exploring a new corpus type may discover a class of hedging language the defaults miss (e.g., contract-specific obligations stated in conditional-future tense, or laboratory-protocol-specific provisional language). The fix is a one-line addition to the domain's `epistemic.py`, not a feature request to the framework. This keeps the framework lean – only the rules that matter to a given domain live in that domain – and keeps domain authors in control of how their corpora are classified.

Table 2: The four pre-built domains in v3.2.0. "Scenarios validated" is the maturity signal – a domain that has been pressure-tested across multiple varied corpora has had its persona, epistemic rules, and entity-type coverage refined in ways one with zero scenarios has not. Schema is static.

Domain	Entities	Relations	Scenarios	Specialty pipeline
drug-discovery	13	22	S1–S6 (six)	RDKit / Biopython molecular validation; structural-node enrichment
contracts	11	11	none yet	—
clinicaltrials	12	10	S7 (one)	Phase-based evidence grading; optional CT.gov enrichment
fda-product-labels	17	16	none yet	Four-level FDA epistemology classifier

3.6 The persona-driven narrator

A separate stage consumes the classified graph and produces a proactive briefing in natural language. The narrator's system prompt is the domain's analyst persona, loaded directly from `workbench/template.yaml`. The input to the narrator is *not* the source documents – it is a structured summary of the classified graph, assembled from `claims_layer.json`: per-status counts, contested-claim pairs with their confidence ranges, prophetic-claim concentrations by entity type, gap signals (entity types with low connection density), and any domain-specific signals the persona is configured to attend to. The narrator emits `epistemic_narrative.md`, structured into Executive Summary, Asserted Claims, Prophetic Claims, Contested Claims, Gaps, and Recommended Follow-Ups sections.

A worked example: on the S6 GLP-1 corpus (drug-discovery domain, 34 documents), the narrator's executive summary identified *61 prophetic claims* concentrated on cardiovascular, neurodegenerative, and metabolic-sub-disorder indications and recommended "temporal stratification of confidence" on the contested `semaglutide INDICATED_FOR obesity` edge (confidence range 0.55–0.97 across sources, where 0.55 instances trace to pre-STEP-1 patent language and 0.97 instances trace to post-approval clinical assertions). The narrator did not see the source documents directly; its claims trace back to graph nodes and relations annotated by the rule engine.

3.7 Persona dual-use: one source, two surfaces

The same persona string is also the system prompt for the workbench's reactive chat panel. When a user asks "which prophetic claims are concentrated on cardiovascular outcomes?" in chat, the answer is generated by the same persona that wrote the briefing, against the same classified graph, with the same vocabulary and decision criteria. This is by design: the proactive briefing and the reactive chat would otherwise drift apart in tone, vocabulary, and analytic framing if they were configured separately. The pattern reduces the surface area of the framework – one YAML field defines the domain's analyst voice – and makes domain extension cheaper: a domain author writes one persona, and both surfaces inherit it.

4 Pre-built Domains

The framework ships with four pre-built domains. Two have been validated against showcase corpora; two are schema scaffolds awaiting their first showcase corpus. Table 2 summarizes their state.

4.1 drug-discovery

The most-developed and most-pressure-tested domain. Thirteen entity types: `COMPOUND`, `PROTEIN`, `DISEASE`, `MECHANISM_OF_ACTION`, `ORGANIZATION`, `PUBLICATION`, `CLINICAL_TRIAL`, `BIOMARKER`, `PHENOTYPE`, `ADVERSE_EVENT`, `PATHWAY`, `REGULATORY_ACTION`, `DOCUMENT`. Twenty-two directed relation types. The schema was consolidated from an earlier 17-entity / 30-relation form by removing low-yield types observed across S1–S5 (entity types and relations that were under-exercised or overlapped with existing types). The S6 GLP-1 corpus rebuild validated the consolidated schema on 34 multi-source documents (10 patents, 24 PubMed papers).

The domain ships with deterministic molecular validation: SMILES strings are validated and canonicalized with RDKit, protein and nucleotide sequences with Biopython, and structured identifiers (CAS registry numbers, NCT numbers, InChIKeys) by regex. Validated identifiers become first-class graph nodes (`CHEMICAL_STRUCTURE`, `PEPTIDE_SEQUENCE`) linked to their parent entities, carrying computed properties as attributes (molecular weight, Lipinski compliance, GC content, isoelectric point). The validation report (`validation_report.json`) is auto-emitted alongside the graph.

The persona is a senior drug-discovery competitive-intelligence analyst, with vocabulary calibrated to therapeutic-area, mechanism-of-action, and regulatory framing. The same persona drives the workbench chat and the narrator briefing on every drug-discovery scenario.

4.2 clinicaltrials

A trial-protocol-aware domain Davidson and Bhatt [2026] for ingestion of `ClinicalTrials.gov` protocol records. Twelve entity types: `Trial`, `TrialPhase`, `Sponsor`, `Compound`, `Intervention`, `Condition`, `Cohort`, `Population`, `Outcome`, `Biomarker`, `Endpoint`, `DOCUMENT`. Ten directed relation types capturing trial structure (`HAS_PHASE`, `SPONSORED_BY`, `TESTS_INTERVENTION`, `ENROLLS_POPULATION`, `MEASURES_OUTCOME`, etc.).

The domain's epistemic module implements phase-based evidence grading: Phase 3 and Phase 4 outcome relations are promoted to `asserted`; Phase 2 retains the rule-engine default; Phase 1 outcomes are demoted to `hypothesized` unless the evidence quote contains explicit quantitative results. This captures a regulatory-grade evidence stratification that a domain-agnostic rule engine cannot infer from text alone.

An optional `--enrich` flag during ingestion fetches additional structured fields from the CT.gov API (NCT-keyed lookup, currently using `urllib` due to a `httpx` 403 issue with the CT.gov upstream that the framework documents in `docs/known-limitations.md`). Enrichment is opt-in because it adds API latency and is unnecessary for protocols that are already self-contained in their JSON exports.

The persona is a clinical trial design and competitive intelligence analyst, with vocabulary attuned to phase, endpoint hierarchy, sponsor strategy, and regulatory pathway.

4.3 contracts

A vendor- and event-contract domain. Eleven entity types covering contract parties (`Party`, `Vendor`, `Sponsor`), structural elements (`Obligation`, `Deadline`, `Deliverable`, `Payment`, `Termination`, `Renewal`), and metadata (`Document`, `Section`). Eleven directed relations capturing contractual structure. The domain has no validated showcase corpus; its inclusion in v3.2.0 demonstrates that the framework supports non-biomedical domains without pipeline modification.

4.4 fda-product-labels

A FDA Structured Product Label (SPL) domain. Seventeen entity types covering drug products (`DrugProduct`, `ActiveIngredient`, `Strength`, `DosageForm`), regulatory metadata (`NDC`, `NDA`,

ANDA, SetID, RxCUI, UNII), label sections (Indication, Contraindication, AdverseReaction, Interaction, Warning, LabTest), and DOCUMENT. Sixteen directed relation types.

The domain's epistemic module implements a four-level FDA epistemology classifier (established, reported, theoretical, asserted) keyed to FDA's recognizable label-text vocabulary ("has been demonstrated," "has been reported," "has been theorized," unmarked Indications-and-Usage assertions). The four levels map onto the shared epistemic taxonomy with finer granularity, allowing the analyst persona to stratify briefings into "regulatory-grade evidence," "post-marketing reports," "mechanistic theory," and "unmarked indications."

The domain has no validated showcase corpus as of v3.2.0; the schema and epistemic module are scaffolds, awaiting a 200–400-document SPL ingestion that GitHub Issue #14 tracks. The persona has been hand-tailored to FDA-canonical citations (SPL set ID, NDA/ANDA, NDC, RxCUI, UNII) in anticipation of the showcase corpus.

4.5 Adding a new domain

A new domain is four files. The framework ships an interactive wizard, `/epistract:domain create <name> <sample-corpus-path>`, that bootstraps the four files in three passes:

1. *Sample analysis.* The wizard reads 5–10 documents from the sample corpus, identifies recurring entity surface forms, and proposes an initial schema (entity types and relation types).

2. *Schema refinement.* The user edits the proposed schema, accepts or rejects entity types, and approves naming standards. The wizard regenerates `domain.yaml`.

3. *Prompt and persona generation.* The wizard generates `SKILL.md` (extraction prompt with examples drawn from the sample corpus), `epistemic.py` (a starter rule engine with shared-taxonomy patterns plus any domain-specific additions the user requested), and `workbench/template.yaml` (a starter persona inferred from the domain's terminology).

The total wall-clock time for a new domain, with a sample corpus in hand, is typically under 15 minutes. The output is immediately runnable – `/epistract:ingest <corpus> --domain <new-name>` works on the freshly generated configuration.

5 Evaluation

The framework has been validated across seven scenarios spanning two of the four pre-built domains. This section presents per-scenario quantitative results, schema coverage analysis, epistemic-layer outputs, and an honest accounting of what the evaluation does and does not establish.

5.1 Scenarios across domains

Table 3 reports the seven validation scenarios. The first six (S1–S6) exercise the drug-discovery domain across varied therapeutic areas; the seventh (S7) is the inaugural clinicaltrials scenario.

The drug-discovery domain has accumulated the most pressure-testing. S1–S5 establish breadth across therapeutic areas; S6 added multi-source corpus assembly (PubMed E-utilities, Google Scholar, Google Patents via SerpAPI) and stress-tested patent-document handling. S7 is the first scenario in a second domain and exercises clinicaltrials' phase-based evidence grading and CT.gov ingestion. Contracts and fda-product-labels do not yet have validated scenarios; they are schema scaffolds.

Table 3: Validation scenarios across active domains, reporting the most recent rebuild of each. "UATs" is the user-acceptance-test pass rate for that scenario. "Rebuild" indicates which framework state produced the reported numbers; the version-progression table for the most heavily rebuilt scenario (S6) appears as Table 4 below. S1–S5 numbers are V2 rebuilds documented in `tests/scenarios/scenario-0{1--5}-*-v2.md`; S6 is the V3 rebuild (Table 4); S7 is the V3.1 launch.

#	Domain / Corpus	Docs	Nodes	Relations	Comm.	UATs	Rebuild
S1	drug-discovery / Neurogenetics (PICALM)	15	183	478	6	4/4	V2
S2	drug-discovery / Oncology (KRAS G12C)	16	140	432	4	5/5	V2
S3	drug-discovery / Rare Disease	15	110	278	4	3/3	V2
S4	drug-discovery / Immuno-Oncology	16	151	440	5	4/4	V2
S5	drug-discovery / CV & Inflammation	15	90	245	4	3/3	V2
S6	drug-discovery / GLP-1 CI	34	278	855	10	6/6	V3
S7	clinicaltrials / GLP-1 Phase 3 (CT.gov)	10	142	395	8	5/5	V3.1
	Drug-discovery (S1–S6)	111	952	2,728	33	25/25	
	Clinicaltrials (S7)	10	142	395	8	5/5	

5.2 Cross-source patent extraction (S6)

The S6 GLP-1 corpus comprises 24 PubMed abstracts and 10 patent filings from five pharmaceutical companies (Novo Nordisk, Eli Lilly, Pfizer, Hanmi Pharmaceutical, Zealand Pharma). The patent component is a deliberate stress test: patents differ structurally from research papers, use heavily prophetic language, and contain molecular identifiers (peptide sequences, CAS registry numbers, InChIKeys) that abstracts do not. The rebuild produced 278 entities and 855 relations on the consolidated 13/22 schema – a higher density than any S1–S5 corpus and the first scenario to exercise multi-source entity merging at scale (semaglutide appears in 17 documents – 4 patents and 13 papers – and is correctly deduplicated to a single node with merged attributes).

Enrichment across rebuilds of the same corpus. The S6 corpus has been rebuilt twice as the framework has matured, holding the source documents constant. Table 4 reports the deltas between the most recent two rebuilds (V2 baseline vs. V3 rebuild), which together exercise the v3.0 pipeline contributions: `chonkie` sentence-aware chunking with overlap (FIDL-03), Pydantic-validated extraction at write time (FIDL-02c), `metadata.domain` round-tripped through the graph (FIDL-06), per-domain validators auto-invoked post-build (FIDL-07), the deterministic Layer 2 `claims_layer.json` artifact, and the persona-driven narrator briefing (`epistemic_narrative.md`). The increases in nodes, edges, and average attributes per node reflect a combination of the framework's Pydantic-strict extraction (no silent drops on schema-misshapen extractions) and the consolidated schema's better fit to the corpus. The much larger increases in *prophetic* (15 → 61) and *contested* (5 → 33) classifications reflect the v3.0 Layer 2 itself: those classifications were simply not produced by earlier rebuilds, because the rule engine and cross-source aggregation had not been built yet. Two artifacts are entirely new in V3 and therefore appear without a baseline: `validation_report.json` and `epistemic_narrative.md`.

5.3 Trial-protocol structure (S7)

The S7 GLP-1 Phase 3 corpus comprises 10 trial protocols from `ClinicalTrials.gov`, hand-selected to span the contemporary GLP-1 receptor agonist clinical landscape (semaglutide STEP and SUSTAIN families, tirzepatide SURPASS family, and orforglipron). The clinicaltrials domain extracted 142 nodes and 395 relations, with the largest entity-type contributions from `Outcome` (65), `Intervention` (16), and `Cohort` (13). The phase-based grading promoted Phase

Table 4: S6 GLP-1 corpus rebuilds, holding the 34-document source corpus constant. "V2" is the rebuild against the earlier framework state (single-layer graph, no narrator); "V3" is the rebuild against the v3.0 framework with the two-layer architecture, the sentence-aware chunker, Pydantic-strict extraction, and the persona-driven narrator. The extractor model also changed (V2 used Opus 4.6, V3 used Sonnet 4.6 via OpenRouter), which is a confound for raw entity / relation counts but does not affect the artifact-level additions.

Metric	**V2**	**V3**	**Δ**
Nodes	193	278	+44%
Edges	619	855	+38%
Communities	9	10	+1
Avg. attributes per node	1.77	3.86	+118%
`prophetic` relations (Layer 2)	15	61	+307%
`contested` relations (Layer 2)	5	33	+560%
`metadata.domain` in `graph_data.json`	—	yes	new (FIDL-06)
`validation_report.json` auto-emitted	—	yes	new (FIDL-07)
`epistemic_narrative.md` auto-emitted	—	yes	new (v3 narrator)

3 outcome relations to `asserted`; the resulting briefing surfaced regulatory-grade head-to-head evidence (e.g., SURPASS-2 tirzepatide-vs-semaglutide) distinct from earlier-phase signals.

5.4 Schema coverage analysis

The drug-discovery schema has gone through one consolidation. The S1–S5 V2 rebuilds ran against an earlier 17-entity / 30-relation form and together exercised 17 of 17 entity types (100%) and 28 of 30 relation types (93%); the two unused relations were `PHOSPHORYLATES` and `METABOLIZED_BY`, which represent specialized biochemical relationships rarely discussed at the abstract level. The schema was subsequently consolidated to 13 entities / 22 relations by removing low-yield types observed across S1–S5; the consolidated schema retained the high-yield types and merged or removed the low-yield ones. The S6 V3 rebuild on the consolidated schema exercised all 13 entity types and 20 of 22 relation types (the two unused being domain-rare structural relations). On the clinicaltrials domain, S7 exercised 11 of 12 entity types (92%, the unused being `Endpoint` as a separate node from `Outcome`) and 9 of 10 relation types (90%).

5.5 Epistemic-layer outputs (S6 case study)

The S6 V3 `claims_layer.json` contained 855 relation classifications: ∼430 `asserted`, 61 `prophetic`, 78 `hypothesized`, 12 `contested` pairs, 2 `contradiction` pairs, 14 `negative` entries, and the remainder `unclassified` or `speculative`. The 61 prophetic claims concentrated on cardiovascular, neurodegenerative, and metabolic-sub-disorder indications – a pattern consistent with patent claim-breadth strategy in the GLP-1 receptor agonist class. The 12 contested pairs included high-stakes edges such as `semaglutide INDICATED_FOR obesity` (confidence range 0.55–0.97 across sources) where pre- and post-STEP-1 trial language coexists in the corpus.

The narrator's executive summary on S6, generated from the classified graph by the senior drug-discovery competitive-intelligence persona, reproduced the prophetic-concentration finding and recommended "temporal stratification of confidence" on the contested obesity-indication edge. It also surfaced an ingestion gap – absence of SURPASS-2 data in the corpus – and recommended that the analyst integrate the trial. The recommendation was correct: SURPASS-2 (Frías et al., NEJM 2021) Frías et al. [2021] was not in the original corpus and is the canonical head-to-head tirzepatide-vs-semaglutide evidence. The narrator did not see the source documents directly; its claims trace back to graph nodes and statuses set by the rule engine.

The narrator output is non-deterministic: re-running the persona-driven stage on the same `claims_layer.json` produces briefings that vary in word choice but stabilize on the same findings (prophetic concentrations, contested edges, gap signals). The deterministic record is `claims_layer.json`; the briefing is its narrative rendering.

5.6 What the evaluation establishes

The evaluation is sufficient to establish the following claims:

- *Domain pluggability is real, not aspirational.* A second active domain (clinicaltrials) ships in v3.1 with its own schema, epistemic rules, and persona, and validates with a first scenario. The pipeline did not require modification.
- *The rule-based epistemic layer produces analyst-relevant signal.* On S6, the layer correctly identified prophetic concentrations and contested edges that match the substantive structure of the GLP-1 corpus.
- *The persona dual-use pattern is coherent.* On both S6 and S7, the chat surface and the briefing surface use the same vocabulary and answer the same questions consistently because they share a persona string.
- *Scenario count is a maturity signal.* Drug-discovery (six scenarios) and clinicaltrials (one scenario) differ in pressure-testing depth, and that difference shows in subjective briefing quality and edge-case handling. The framework exposes this gap honestly rather than masking it.

5.7 What the evaluation does not establish

The evaluation does *not* establish:

- *Cross-scenario auto-learning.* The framework does not currently fold lessons from prior scenario runs into subsequent runs. Refinements (improved persona vocabulary, refined epistemic rules, schema consolidation) are human-mediated: a contributor reads the narrator output, identifies a missed pattern, edits the domain configuration, and commits. The README and the framework treat scenario count as a track record, not as evidence of compounding learning. GitHub Issue #15 tracks the aspirational mechanism.
- *Universal coverage of FDA labels or contracts.* Two of the four pre-built domains lack validated showcase corpora. Their schemas are designed but not pressure-tested. GitHub Issue #14 tracks the fda-product-labels showcase corpus.
- *Narrator output reliability for clinical decisions.* The narrator is a research tool, not a regulatory or clinical decision-making system. Its outputs are hypotheses requiring expert review, and the system documents this caveat both in-product and in the README.

These caveats are not afterthoughts. They are first-class limitations the framework documents explicitly and tracks in the issue queue. Section 8 expands on the responsible-use boundary.

6 Relation to similarity-based KG construction

For readers approaching this work from the GraphRAG Edge et al. [2024] or embedding-similarity tradition of knowledge graph construction, this section sketches where Epistract sits relative to that work. It is meant as orientation, not as a structural argument; the framework is what it is, and a reader who skips this section loses no part of the contribution.

Table 5: Where Epistract sits relative to similarity-based KG construction.

Dimension	**Similarity-based KG (e.g., GraphRAG)**	**Epistract**
Relationship basis	Embedding similarity	LLM comprehension + categorical epistemic typing
Layers	One	Two (brute facts + epistemic superdomain)
Schema	Generic NER	Domain-specific: four pre-built domains, 11–17 ent / 10–22 rel each
Relation typing	Untyped	Typed, directional, with per-relation epistemic status
Evidence	None	Verbatim source passage, confidence, classification rule, doc-type inference
Validation	None	Domain-specific: RDKit, Biopython, NCT/CAS for drug-discovery; opt-in CT.gov enrichment for clinicaltrials
Briefing	None	Auto-generated narrator briefing per run
Chat surface	None	Workbench chat, same persona as briefing
Persona	N/A	One YAML string per domain, drives both briefing and chat
Source types	Single-corpus	PubMed, Google Scholar, Google Patents, ClinicalTrials.gov, user-provided PDFs
Reproducibility	Code on GitHub	Plugin + committed artifacts; reproducible epistemic layer; non-deterministic narrator stable in structure
Documented gaps	None enumerated	Issues #14 (fda showcase corpus), #15 (cross-scenario auto-learning), `docs/known-limitations.md`

Comprehension over similarity. Similarity-based KG construction Edge et al. [2024] extracts edges from embedding proximity over text chunks: two entities that frequently co-occur in similar contexts get connected. The resulting graph is useful for retrieval and clustering but does not capture mechanistic, causal, or regulatory relationships – the kinds of relationships that matter for biomedical, clinical, and regulatory analytical work. Epistract's Layer 1 is built on a different substrate: an LLM agent reads a full document with domain-specific extraction guidance and produces typed, directional, evidence-backed relations. The graph that results captures meaning rather than co-occurrence, and supports queries that similarity-based graphs cannot serve – "what compounds inhibit KRAS G12C?" returns a list of compounds with explicit `INHIBITS` relationships, not a list of nearest neighbors in an embedding space.

The epistemic dimension. Beyond typed relations, Epistract adds a categorical *epistemic* dimension that similarity-based and most comprehension-based extractors lack. A relation between *semaglutide* and *cardiovascular disease* is not by itself a useful unit of analysis: the relation could be asserted by post-approval clinical evidence, prophesied by patent claim breadth, or hypothesized in research speculation. Layer 2 makes this distinction explicit, by category, on every edge. The analyst querying the graph can stratify by status; the persona-driven briefing surfaces patterns at the corpus level ("61 prophetic claims concentrated on cardiovascular indications").

From extraction artifact to analyst surface. Many KG construction frameworks treat the graph as the product. Epistract treats the graph as the input to two analyst-shaped surfaces: a proactive briefing auto-generated per run, and a reactive chat both grounded in the same domain persona. The framework is shaped around how a scientist would actually use a knowledge graph – not just visualize it, but interrogate it – and the persona dual-use pattern is a deliberate choice to keep those two interrogation modes coherent.

Honest accounting. The framework is explicit about what it does and does not do. The README, the in-product documentation, and Section 5 of this paper distinguish "track record" (a domain has been pressure-tested across multiple scenarios) from "compounding learning" (the framework would automatically benefit from prior runs). The framework does the first; the second is aspirational and tracked as Issue #15. The fda-product-labels domain ships with a schema and a persona but no validated showcase corpus yet; that is tracked as Issue #14. These caveats are first-class limitations, documented in the issue queue rather than buried in footnotes.

7 Human-AI Collaboration Process

The framework was built through intensive interactive Claude Code sessions in which the human drove scientific strategy and acceptance criteria, and the AI drove implementation, parallelization, and documentation. The collaboration is bidirectional: the human curates the corpora, defines what counts as a passing user-acceptance test, and corrects framing drifts; the AI implements, tests in parallel across multiple scenarios, and surfaces inconsistencies for review. This section makes three observations specific to how that collaboration shaped the current framework.

7.1 The GSD workflow as methodology

The framework was built over twenty phases under a structured planning workflow ("Get Shit Done," GSD), in which each phase begins with a written goal and acceptance criteria, proceeds through a discussion-and-research stage, lands a plan, executes against atomic commits, and verifies goal achievement before closing. The workflow's artifacts (`ROADMAP.md`, `REQUIREMENTS.md`, per-phase `PLAN.md` and `VERIFICATION.md`) are local-only by design (gitignored under `.planning/`) but they shape the public commit history: each commit message names its phase and the requirement it discharges, producing an auditable trail from goal to implementation to verification.

The twenty phases that produced the v3.0 framework grouped into three arcs. Phases 1–10 were foundational: domain configuration, document ingestion, entity extraction and graph construction, cross-reference analysis, the interactive dashboard, repo reorganization, the testing framework, the domain-creation wizard, consumer decoupling, and documentation refresh. Phase 11 was end-to-end scenario validation, the V2 rebuilds of S1–S5 reported in Table 3. Phases 12–20 produced the v3.0 graph-fidelity-and-honest-limits arc: structural-biology document signatures (Phase 12), extraction pipeline reliability and Pydantic-strict write-time validation (Phase 13, FIDL-02c), sentence-aware chunk overlap via `chonkie` (Phase 14, FIDL-03; the chunking strategy described in Section 2), format discovery parity (Phase 15, FIDL-04), wizard sample-window expansion (Phase 16, FIDL-05), domain awareness in downstream consumers (Phase 17, FIDL-06), per-domain epistemic and validator extensibility (Phase 18, FIDL-07; the Layer 2 customization paths described in Section 3), wizard and CLI ergonomics (Phase 19, FIDL-08), and the public-facing capacity-and-limits documentation (Phase 20, FIDL-09). Each requirement was scoped tightly enough to verify in isolation; their composition produced the

v3.0 release. Phase 21 added the clinicaltrials domain (v3.1), and Phase 22 added the fda-product-labels domain (v3.2).

7.2 External contribution: the controlled-port pattern

The clinicaltrials domain (Section 4.2) and the fda-product-labels domain (Section 4) were initially contributed by Chris Davidson via pull request. In both cases the original contribution arrived against a stale fork (the contributor's main branch lagged the upstream main by a release or more), and a direct rebase or merge would either lose context or pull in unrelated drift. The framework adopted a *controlled-port pattern*: rather than rebasing the contributor's branch onto upstream main, the maintainer creates a new branch off current upstream main, hand-ports the substantive contribution file-by-file with attribution preserved via Git's `Co-Authored-By` trailer, and reviews each ported file against current conventions. This avoided two recurring failure modes (contributor-introduced `.planning/` files leaking into the public repo, and stub configurations replacing hand-tailored files) and preserved authorship attribution. The pattern is documented in repository memory for future contributions.

7.3 Honesty as a development practice

Several v3 commits are corrections rather than additions. The README was found to overclaim "domain knowledge compounds across scenarios – patterns generalize, edge cases surface, the analyst persona learns what to flag," implying an automatic-learning mechanism that the framework does not implement. The correction reframed scenario count as a track record (pressure-testing depth) rather than an auto-learning signal, and opened GitHub Issue #15 to track the aspirational mechanism. The visible commit (`a8a91db`) does not add a feature; it walks back a marketing claim. The collaboration model includes the human's role in catching such drifts – the AI alone is unlikely to identify when its own framing has shifted from "claim" to "promise."

The pattern recurs in three places: the README compounding correction, the demo-script accuracy checklist (every claim mapped to a verifiable codebase source), and this paper's Section 5 ("What the evaluation does not establish"). Each is an artifact of treating documentation honesty as a first-class development task rather than as polish.

8 Availability, Reproducibility, and Limitations

8.1 Availability

Epistract is released as open-source software under the MIT license at https://github.com/usathyan/epistract. Installation is two commands: `/plugin install epistract` (within Claude Code) and `/epistract:setup` (which installs the Python pipeline, sift-kg, RDKit, and Biopython into an isolated virtual environment). The runtime requirements are Python 3.11 or later, Claude Code, and (for the chat and narrator stages) a credential for one of three supported LLM gateways: Azure AI Foundry, Anthropic direct, or OpenRouter, resolved in that priority order.

8.2 Reproducibility

All seven validation scenarios are reproducible from the public repository. Each scenario directory under `tests/corpora/` contains the source documents (or the SerpAPI / E-utilities query parameters used to assemble them, where licensing prohibits redistribution), the extraction outputs in JSON, the constructed graph data, the classified `claims_layer.json`, and the

narrator's `epistemic_narrative.md`. The classified layer is bit-identical across runs given identical Layer 1 input. The narrator output is non-deterministic but stable in structure (executive summary, prophetic, contested, gaps, follow-ups) and findings (run-to-run word-choice variance, run-to-run finding-set stability).

The plugin architecture provides a versioning benefit: the deployed framework version is recorded in every output artifact's metadata, and committed test baselines serve as regression anchors across releases. The v3.2.0 release notes name the four shipped domains, the seven validated scenarios, and the gaps tracked in Issues #14 and #15.

8.3 Documented gaps

The framework documents three classes of gap explicitly:

Issue #14: fda-product-labels showcase corpus. The domain ships with schema, epistemic rules, and persona, but no validated showcase corpus. A 200–400-document SPL ingestion is in flight with the contributor.

Issue #15: cross-scenario knowledge persistence. The framework does not currently fold lessons from prior scenario runs into subsequent runs. Five candidate mechanisms for compounding knowledge across scenarios are scoped (cross-scenario entity canonicalization, domain-level aggregate graphs, persona few-shot from prior briefings, epistemic-rule mining, cross-scenario claim arbitration), with the first identified as the most tractable v3.3 candidate. This is aspirational; no work has begun.

`docs/known-limitations.md`. A running ledger of operational issues that constrain the framework: `httpx` 403 responses against `ClinicalTrials.gov` (use `urllib` instead), agent-tool model overrides failing silently when the parent runs a 1M-context Sonnet variant, workbench chat `max_tokens` truncation at 4096, and others. Each entry is logged with a workaround and a tracking reference.

8.4 Responsible use

Epistract is a research tool, not a clinical or regulatory decision-making system. The knowledge graphs it produces are derived from LLM extraction (probabilistic) and rule-based epistemic classification (deterministic, but rule-bounded). The narrator briefings are LLM-generated and non-deterministic. Every output should be treated as a hypothesis requiring expert review before informing any scientific, clinical, or regulatory decision. The framework explicitly discloses this caveat in the README, in the in-product workbench, and in this paper.

The persona pattern is a particular focus of the responsible-use boundary. A senior drug-discovery competitive-intelligence analyst is a *voice*; the framework does not certify that any briefing meets professional standards, and users in pharmaceutical and healthcare settings should apply institutional review and domain expert oversight before acting on any framework output.

8.5 Community contribution

The framework's design invites contribution at four levels: new evaluation scenarios with curated corpora that pressure-test existing domains; schema extensions that introduce entity or relation types; new pre-built domains beyond the four shipped (the `/epistract:domain` wizard reduces the scaffolding cost to under 15 minutes for a new domain with a sample corpus); and external database connectors for entity enrichment (PubChem, ChEMBL, UniProt,

ClinicalTrials.gov). Contribution patterns are documented in `docs/ADDING-DOMAINS.md`; the controlled-port pattern (Section 7) is documented for cross-version contributions.

9 Conclusion

Epistract is the research-consolidation layer for scientists and computational analysts working inside Claude Code. Deep research happens upstream – through Claude Code's built-in web tools, MCP-mediated source connectors, and the broader plugin ecosystem – producing a body of documents that answers a research direction. Epistract takes that body and consolidates it into a two-layer knowledge graph, an epistemic classification of every relation, and an analyst briefing, from which the scientist answers a bundle of specific questions and iterates as their understanding of the consolidated body grows. Three architectural choices give the framework its current shape. First, a categorical *epistemic super-domain layer* annotates every relation with a status (`asserted`, `prophetic`, `hypothesized`, `contested`, `contradiction`, `negative`, `speculative`) using a deterministic rule engine that combines hedging-pattern matching with document-type inference. Second, the framework is *domain-agnostic*: a domain is four configuration files (schema, prompt, epistemic rules, persona), and the core pipeline does not need to be modified to add one. Third, the analyst persona is reified as a *single source of truth* per domain that drives both a proactive narrator briefing and a reactive workbench chat, keeping the two surfaces coherent.

The framework ships with four pre-built domains. Two – drug-discovery (six scenarios, 111 documents) and clinicaltrials (one scenario, 10 protocols) – are validated. Two – contracts and fda-product-labels – are schema scaffolds awaiting their first showcase corpus. The framework documents this gap honestly and tracks it in the public issue queue rather than presenting all four domains as if at parity.

The framework's contribution is methodological as much as architectural. Treating the epistemic dimension as a first-class layer changes what a knowledge graph is for: not a structural artifact a human reads in a visualizer, but the input to an analyst persona that synthesizes briefings and answers reactive questions. The persona dual-use pattern – one YAML string, two surfaces – collapses what would otherwise be two separately-maintained system prompts into one and prevents drift. The controlled-port pattern (Section 7) shows that the same architectural choices that make the framework pluggable also make external contribution tractable: a domain is a directory, not a patch against the core pipeline.

Scope: small-to-medium corpora that fit a Claude Code session. The design point is interactive analytical sessions in which a scientist builds a bespoke graph to answer a specific question, iterates by adding documents and re-classifying, and follows up in chat. The seven validated scenarios range from 10 to 34 documents (Section 5); the framework has not been tested at corpora of hundreds or thousands of documents and the upper bound at which the workbench chat or narrator briefing degrades is undetermined. The honest position is to scope the framework to interactive analytical work and to provide an explicit escape hatch for scale: `/epistract:export` dumps the graph to GraphML, CSV, SQLite, or JSON for ingestion into Neo4j-class persistent stores, where larger-scale or multi-team analysis can continue outside the Claude Code session.

Future directions. Several directions remain open. Cross-scenario knowledge persistence (Issue #15) would let a domain compound across runs – a canonical entity registry, then aggregate graphs, then learned rules – and is the most architecturally meaningful next step. The fda-product-labels showcase corpus (Issue #14) will exercise the four-level FDA epistemology classifier under real volume. *MCP-mediated data-source connectors* are an aspirational direction worth highlighting: today a scientist assembles a corpus by ad-hoc query against PubMed,

Google Scholar, ClinicalTrials.gov, or SerpAPI; tomorrow the same corpus could be assembled *on demand* by Claude Code MCP servers exposing those sources (and proprietary sources within institutional environments) as structured tools available to the framework. This would close the loop between question and corpus: the scientist asks "what does the corpus need to look like to answer this?" and an MCP-enabled framework assembles a small corpus inside the session that fits the question. Additional domains beyond the four shipped will test whether the wizard's 15-minute claim holds across diverse corpora. And the integration with regulatory and clinical workflows – the responsible-use boundary – needs continuing attention as the framework's outputs move from research artifacts to decision inputs.

The framework, all corpora, all artifacts, and this paper are reproducible from https://github.com/usathyan/epistract under the MIT license.

References

chonkie: Sentence-aware text chunking for Python. https://pypi.org/project/chonkie/, 2024. Python package providing `SentenceChunker` with character-tokenizer support and configurable overlap; used by Epistract's per-document agents for sentence-boundary-preserving chunking with last-3-sentence overlap.

Anthropic. Claude code: Agentic coding in your terminal. https://www.anthropic.com/claude-code, 2024.

Alessandro Ceresa. sift-kg: A knowledge graph construction toolkit. https://github.com/siftkgc/sift-kg, 2024.

Peter J A Cock, Tiago Antao, Jeffrey T Chang, Brad A Chapman, Cymon J Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J L de Hoon. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.

Chris Davidson and Umesh Bhatt. The clinicaltrials domain for epistract. Pull request #11, https://github.com/usathyan/epistract/pull/11, 2026. Initial contribution of the clinicaltrials domain (schema, epistemic rules, persona) and the S7 GLP-1 Phase 3 showcase corpus.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Dolan. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Juan P Frías, Melanie J Davies, Julio Rosenstock, Federico C Pérez Manghi, Laura Fernández Landó, Bret K Bergman, Bing Liu, Xiaoyu Cui, and Katelyn Brown. Tirzepatide versus semaglutide once weekly in patients with type 2 diabetes. *New England Journal of Medicine*, 385(6):503–515, 2021.

Greg Landrum. RDKit: Open-source cheminformatics. https://www.rdkit.org, 2024.

Giancarlo Perrone. pyvis: Interactive network visualization in Python. https://github.com/WestHealth/pyvis, 2018.